

LAPORAN PRAKTIKUM STRUKTUR DATA

Single Linked List

Disusun Oleh :

Aliifah Felda Mufarrihati Salwaa

2511531011

Dosen Pengampu :

Dr. Wahyudi, M.T

Asisten Praktikum :

M Galid Avaro



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Laporan praktikum ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum Struktur Data yang membahas tentang Single Linked List. Melalui laporan ini penulis dapat memahami materi praktikum secara mendalam dan dapat lebih teliti, teratur, serta memiliki kemampuan yang baik dalam penulisan kode program sesuai kaidah akademik. Sehingga laporan ini dapat menjadi sarana belajar, dokumentasi kegiatan, dan referensi praktikum berikutnya.

Penulis menyadari bahwa laporan ini masih memiliki banyak kekurangan, baik dari isi maupun penyajiannya. Oleh karena itu, saran dan kritik sangat penulis harapkan guna untuk menyempurnakan laporan berikutnya.

Padang, 04 Mei 2026

Aliifah Felda Mufarrihati Salwaa

2511531011

DAFTAR ISI

KATA PENGANTAR.....	i
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang Praktikum	3
1.2 Tujuan Praktikum	3
1.3 Manfaat Praktikum	3
BAB II PEMBAHASAN	4
2.1. Dasar Teori	4
2.2. Class QueueArray_2511531011.....	5
2.3. QueueArrayDriver_2511531011.....	5
2.4. ReverseData_2511531011	8
2.5. IterasiQueue_2511531011	11
2.6. QueueLinkedList_2511531011	Error! Bookmark not defined.
2.7. Tugas Pekan 4	12
BAB III KESIMPULAN.....	18
3.1. Kesimpulan.....	18
3.2. Saran	18
DAFTAR PUSTAKA	19

BAB I

PENDAHULUAN

1.1 Latar Belakang Praktikum

Setelah mempelajari salah satu implementasi ADT yakni array list, stack, dan queue pada praktikum sebelumnya, kita juga harus mengetahui tentang penerapan ADT yang lain. Karena banyaknya implementasi ADT, kita harus dapat menguasai setiap implementasinya agar dapat membangun sebuah program efektif dan efisien, namun tetap sesuai dengan keinginan kita.

Oleh karena itu, pada praktikum kali ini kita akan mempelajari dan memahami tentang penerapan ADT yang lainnya yakni Single Linked List. Single Linked List merupakan struktur data dimana setiap elemen terhubung dengan elemen berikutnya. Setiap elemen (node) memiliki field yang berisi data dan field yang berisi pointer ke node berikutnya.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah sebagai berikut:

1. Memahami tentang implementasi ADT yakni Single Linked List.
2. Mengetahui kelebihan dan kekurangan Single Linked List dalam program.
3. Dapat membangun sebuah kode program dengan Single Linked List.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini adalah sebagai berikut:

1. Mampu mengimplementasikan Single Linked List dalam pembuatan program agar lebih efektif dan efisien.
2. Memahami penggunaan Single Linked List dalam program.

BAB II

PEMBAHASAN

2.1. Dasar Teori

Linked list merupakan struktur data linier dimana elemen-elemen yang disebut node disimpan secara berurutan tetapi tidak harus di lokasi memori yang berdekatan seperti array. Setiap node pada linked list memiliki 2 komponen utama, yakni: data (nilai node) dan pointer (penunjuk node berikutnya).

Linked list ini terbagi pada 2 jenis, yakni;

1. Single Linked List: setiap node hanya memiliki satu pointer menuju node berikutnya dan node terakhir menunjuk pada null
2. Double Linked List: setiap node memiliki dua pointer, yakni untuk menunjukkan node berikutnya dan untuk menunjukkan node sebelumnya.

Single Linked List merupakan Linked List yang paling sederhana. Setiap node hanya memiliki pointer ke node selanjutnya. Tidak ada pointer ke node sebelumnya sehingga pencarian hanya dapat dilakukan dalam satu arah.

Selain karena Single Linked List lebih sederhana, penggunaan Single Linked List juga didasari atas kebutuhan dan keunggulan/kekurangan yang dimiliki oleh Single Linked List. Berikut adalah keunggulan Single Linked List:

- Ukuran dinamis: dapat bertambah dan berkurang secara dinamis sehingga lebih hemat memori
- Penyisipan/penghapusan yang efisien

Sementara itu, berikut adalah kekurangan dari Single Linked List:

- Akses berurutan sehingga hanya dapat melintasi satu arah
- Tidak ada akses langsung untuk node sehingga harus menelusuri dari kepala/head.

2.2. Class NodeSLL_2511531011

```
package pekan5_2511531011;

public class NodeSLL_2511531011 {
    //node bagian data
    int data_1011;
    //pointer ke node berikutnya
    NodeSLL_2511531011 next_1011;
    //konstruktor menginisialisasi node dengan data
    public NodeSLL_2511531011(int data_1011)
    {
        this.data_1011 = data_1011;
        this.next_1011 = null;
    }
}
```

Kode program 2.1: NodeSLL_2511531011

Class ini merupakan konstruktor untuk membuat single linked list. Disini kita menggunakan data_1011 untuk menunjukkan data yang tersimpan dan next_1011 sebagai pointer di set null agar tidak menunjuk kemanapun.

2.3. TambahSLL_2511531011

```
package pekan5_2511531011;

public class TambahSLL_2511531011 {
    public static NodeSLL_2511531011
    insertAtFront_2511531011(NodeSLL_2511531011 head_1011, int value_1011) {
        NodeSLL_2511531011 new_node_1011 = new
        NodeSLL_2511531011(value_1011);
        new_node_1011.next_1011 = head_1011;
        return new_node_1011;
    }
    //fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511531011
    insertAtEnd_2511531011(NodeSLL_2511531011 head_1011, int value_1011) {
        //buat sebuah node dengan sebuah nilai
        NodeSLL_2511531011 newNode_1011 = new
        NodeSLL_2511531011(value_1011);
        //jika list kosong maka node jadi head
        if (head_1011 == null) {
            return newNode_1011;
        }
        //simpan head ke variabel sementara
        NodeSLL_2511531011 last_1011 = head_1011;
        //telusuri ke node akhir
        while (last_1011.next_1011 != null) {
            last_1011 = last_1011.next_1011;
        }
    }
}
```

```

    }
    //ubah pointer
    last_1011.next_1011 = newNode_1011;
    return head_1011;
}
static NodeSLL_2511531011 GetNode_2511531011(int data_1011) {
    return new NodeSLL_2511531011(data_1011);
}
static NodeSLL_2511531011 insertPos_2511531011(NodeSLL_2511531011
headNode_1011, int position_1011, int value_1011) {
    NodeSLL_2511531011 head_1011 = headNode_1011;
    if (position_1011 < 1)
        System.out.print("Invalid Position");
    if (position_1011 == 1) {
        NodeSLL_2511531011 new_node_1011 = new
NodeSLL_2511531011(value_1011);
        new_node_1011.next_1011 = head_1011;
        return new_node_1011;
    } else {
        while (position_1011-- != 0) {
            if (position_1011 == 1) {
                NodeSLL_2511531011 newNode_1011 =
GetNode_2511531011(value_1011);
                newNode_1011.next_1011 =
headNode_1011.next_1011;
                headNode_1011.next_1011 = newNode_1011;
                break;
            }
            headNode_1011 = headNode_1011.next_1011;
        }
        if (position_1011 != 1)
            System.out.print("Posisi diluar jangkauan");
    }
    return head_1011;
}
public static void printList_2511531011(NodeSLL_2511531011
head_1011) {
    NodeSLL_2511531011 curr_1011 = head_1011;
    while (curr_1011.next_1011 != null) {
        System.out.print(curr_1011.data_1011+"-->");
        curr_1011 = curr_1011.next_1011;
    }
    if (curr_1011.next_1011==null) {
        System.out.print(curr_1011.data_1011);
    }
    System.out.println();
}
public static void main(String [] args) {
    //buat linkedlist 2->3->4->5->6
    NodeSLL_2511531011 head_1011 = new
NodeSLL_2511531011(2);
    head_1011.next_1011 = new NodeSLL_2511531011(3);
    head_1011.next_1011.next_1011 = new
NodeSLL_2511531011(5);
    head_1011.next_1011.next_1011.next_1011 = new
NodeSLL_2511531011(6);
    //cetak list asli

```

```

        System.out.print("Senarai Berantai Awal: ");
        printList_2511531011(head_1011);
        //tambahkan node baru di depan
        System.out.print("tambah 1 simpul di depan: ");
        int data_1011 = 1;
        head_1011 = insertAtFront_2511531011(head_1011,
data_1011);

        //cetak update list
        printList_2511531011(head_1011);
        //tambahkan baru di belakang
        System.out.print("tambah 1 simpul di belakang: ");
        int data2_1011 = 7;
        head_1011 = insertAtEnd_2511531011(head_1011,
data2_1011);

        //cetak update list
        printList_2511531011(head_1011);
        System.out.print("tambah 1 simpul ke data 4: ");
        int data3_1011 = 4;
        int pos_1011 = 4;
        head_1011 = insertPos_2511531011(head_1011,
pos_1011, data3_1011);
        //cetak update list
        printList_2511531011(head_1011);
    }
}

```

Kode program 2.2: TambahSLL_2511531011

Pada kode program ini kita akan mengetahui dan mencoba bagaimana cara untuk membuat dan menambah data dalam struktur data single linked list. Disini kita menggunakan banyak method, yakni:

1. insertAtFront_2511531011: method ini digunakan untuk menambahkan node pada bagian depan SLL dengan menjadikan node baru sebagai head dan head lama menjadi node selanjutnya setelah node baru. contoh: 2->3 dimasukkan 1 diawal menjadi 1->2->3 (menjadikan 1 sebagai head dan 2/head lama menjadi node setelah head baru).
2. insertAtEnd_2511531011: method ini digunakan untuk menambahkan node pada bagian akhir single linked list. Menambahkan node dilakukan dengan mengecek apakah list kosong, jika iya maka node baru menjadi head. Jika tidak, lakukan loop untuk mencari node terakhir dan menambahkan node pada bagian belakang list.
3. GetNode_2511531011: method ini digunakan untuk mendapatkan node yang diinginkan.

4. insertPos_2511531011: method ini digunakan untuk menyisipkan node pada posisi tertentu. Jika posisi target adalah 1, ganti head dengan node baru. Jika posisi target selain 1, geser node baru sampai menemukan posisi target dan jadikan node baru menunjuk setelahnya dan node sebelumnya menunjuk node baru.
5. printList_2511531011: method ini digunakan untuk mencetak seluruh data yang ada dalam list dengan melakukan loop hingga node terakhir.

Selain method yang digunakan, kita juga menggunakan class main untuk eksekusi langsung program. Pada class main ini kita akan memanggil class NodeSLL untuk membuat node sesuai yang kita inginkan. Kemudian lakukan cetak pada list node yang telah dimasukkan di awal. Kita juga dapat menambahkan node di awal dengan method insertAtFront, menambah node di akhir dengan method insertAtEnd, menambah node ditengah dengan method insertPos. Kita juga dapat melihat setiap perubahan yang terjadi pada list dengan menambahkan method printList setelah method untuk menambahkan node.

Berikut adalah salah satu contoh output yang kita dapatkan dengan menggunakan banyak method tambahSLL yang telah kita sediakan.

Output :

```
<terminated> TambahSLL_2511531011 [Java Application] D:\Eclipse\workspace
Senarai Berantai Awal: 2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

2.4. HapusSLL_2511531011

```
package pekan5_2511531011;

public class HapusSLL_2511531011 {
    //fungsi untuk menghapus head
    public static NodeSLL_2511531011
deleteHead_2511531011(NodeSLL_2511531011 head_1011) {
        //jika SLL kosong
        if (head_1011 == null)
            return null;
        //pindahkan head ke node berikutnya
```

```

        head_1011 = head_1011.next_1011;
        //return head baru
        return head_1011;
    }
    //fungsi menghapus node terakhir SLL
    public static NodeSLL_2511531011 removeLastNode_2511531011
(NodeSLL_2511531011 head_1011) {
        //jika list kosong, return null
        if (head_1011 == null) {
            return null;
        }
        //jika list satu node, hapus node dan return null
        if (head_1011.next_1011 == null) {
            return null;
        }
        //temukan node terakhir ke dua
        NodeSLL_2511531011 secondLast_2511531011 = head_1011;
        while (secondLast_2511531011.next_1011.next_1011 != null) {
            secondLast_2511531011 =
secondLast_2511531011.next_1011;
        }
        //hapus node terakhir
        secondLast_2511531011.next_1011 = null;
        return head_1011;
    }
    //fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511531011
deleteNode_2511531011(NodeSLL_2511531011 head_1011, int position_1011) {
        NodeSLL_2511531011 temp_1011 = head_1011;
        NodeSLL_2511531011 prev_1011 = null;
        //jika linked list null
        if (temp_1011 == null)
            return head_1011;
        //kasus 1: head dihapus
        if (position_1011 == 1) {
            head_1011 = temp_1011.next_1011;
            return head_1011;
        }
        //kasus 2: menghapus node di tengah
        //telusuri ke node yang dihapus
        for (int i = 1; temp_1011 != null && i < position_1011;
i++) {
            prev_1011 = temp_1011;
            temp_1011 = temp_1011.next_1011;
        }
        //jika ditemukan, hapus node
        if (temp_1011 != null) {
            prev_1011.next_1011 = temp_1011.next_1011;
        } else {
            System.out.print("Data tidak ada");
        }
        return head_1011;
    }
    //fungsi mencetak SLL
    public static void printList_2511531011(NodeSLL_2511531011
head_1011) {
        NodeSLL_2511531011 curr_1011 = head_1011;

```

```

        while (curr_1011.next_1011 != null) {
            System.out.print(curr_1011.data_1011 + "-->");
            curr_1011 = curr_1011.next_1011;
        }
        if (curr_1011.next_1011 == null) {
            System.out.print(curr_1011.data_1011);
        }
        System.out.println();
    }
    //kelas main
    public static void main (String [] args) {
        //buat SLL 1->2->3->4->5->6->null
        NodeSLL_2511531011 head_1011 = new NodeSLL_2511531011(1);
        head_1011.next_1011 = new NodeSLL_2511531011(2);
        head_1011.next_1011.next_1011 = new NodeSLL_2511531011(3);
        head_1011.next_1011.next_1011.next_1011 = new
NodeSLL_2511531011(4);
        head_1011.next_1011.next_1011.next_1011.next_1011 = new
NodeSLL_2511531011(5);
        head_1011.next_1011.next_1011.next_1011.next_1011.next_1011
= new NodeSLL_2511531011(6);
        //cetak list awal
        System.out.println("List awal: ");
        printList_2511531011(head_1011);
        //hapus head
        head_1011 = deleteHead_2511531011(head_1011);
        System.out.println("List setelah head dihapus: ");
        printList_2511531011(head_1011);
        //hapus node terakhir
        head_1011 = removeLastNode_2511531011(head_1011);
        System.out.println("List setelah simpul terakhir dihapus:
");
        printList_2511531011(head_1011);
        //deleting node at position 2
        int position_1011 = 2;
        head_1011 = deleteNode_2511531011(head_1011,
position_1011);
        //print list after deletion
        System.out.println("List setelah posisi 2 dihapus: ");
        printList_2511531011(head_1011);
    }
}

```

Kode program 2.3: HapusSLL_2511531011

Dalam kode program ini kita juga akan membuat beberapa method untuk membantu kita dalam menghapus sebuah node. Berikut adalah node yang akan digunakan:

1. deleteHead_2511531011: untuk menghapus node pertama dan mengubah nilai head dengan node setelahnya.

2. `removeLasNode_2511531011`: untuk menghapus node terakhir dengan cara mencari node terakhir, menghapus node terakhir, dan mengubah node terakhir dengan node sebelumnya.
3. `deleteNode_2511531011`: untuk menghapus node pada indeks tertentu. Jika tujuan adalah node 1, maka hapus dan jadikan node berikutnya head. Jika selain node pertama, telusuri node dan hapus.
4. `printList_2511531011`: untuk print seluruh node yang ada pada list.

Disini kita juga memiliki class main untuk menjalankan program menggunakan seluruh method yang telah kita gunakan. Disini kita akan memanggil class `NodeSLL` untuk membuat node baru dan memasukkan data sesuai yang kita inginkan. Kemudian kita bisa menghapus node dengan menggunakan seluruh method yang telah kita gunakan sesuai dengan fungsinya masing-masing.

Output :

```

List awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5

```

2.5. PencarianSLL_2511531011

```

package pekan5_2511531011;

public class PencarianSLL_2511531011 {
    static boolean searchKey_2511531011 (NodeSLL_2511531011
head_1011, int key_1011) {
        NodeSLL_2511531011 curr_1011 = head_1011;
        while (curr_1011 != null) {
            if (curr_1011.data_1011 == key_1011)
                return true;
            curr_1011 = curr_1011.next_1011;
        }
        return false;
    }
    public static void traversal_2511531011 (NodeSLL_2511531011
head_1011) {
        //mulai dari head
        NodeSLL_2511531011 curr_1011 = head_1011;
    }
}

```

```

        //telusuri sampai pointer null
        while (curr_1011 != null) {
            System.out.print(" " + curr_1011.data_1011);
            curr_1011 = curr_1011.next_1011;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeSLL_2511531011 head_1011 = new NodeSLL_2511531011(14);
        head_1011.next_1011 = new NodeSLL_2511531011(21);
        head_1011.next_1011.next_1011 = new NodeSLL_2511531011(13);
        head_1011.next_1011.next_1011.next_1011 = new
NodeSLL_2511531011(30);
        head_1011.next_1011.next_1011.next_1011.next_1011 = new
NodeSLL_2511531011(10);
        System.out.print("Penelusuran SLL : ");
        traversal_2511531011(head_1011);
        //data yang akan dicari
        int key_1011 = 30;
        System.out.print("Cari data " + key_1011 + " = ");
        if (searchKey_2511531011(head_1011, key_1011))
            System.out.println("ketemu");
        else
            System.out.println("tidak ada");
    }
}

```

Kode program 2.4: PencarianSLL_2511531011

Pada kode program ini kita juga membuat beberapa method untuk mencari node dengan ketentuannya masing-masing, yakni:

1. searchKey_2511531011: untuk mencari nilai tertentu di dalam node dengan melakukan mencari dari node awal.
2. traversal_2511531011: untuk menampilkan seluruh isi list.\

Pada kode program ini kita juga memiliki class main untuk eksekusi program menggunakan method yang telah kita buat. Berikut adalah output dari class main:

Output :

```

<terminated> PencarianSLL_2511531011.java
Penelusuran SLL : 14 21 13 30 10
Cari data 30 = ketemu

```

2.6. Tugas Pekan 4

A. Pasien_2511531011

```
package pekan5_2511531011;

public class Pasien_2511531011 {
    //atribut pasien
    String namaPasien_1011;
    String keluhan_1011;
    int nomor_1011;

    Pasien_2511531011 next_1011;

    //konstruktor pasien
    Pasien_2511531011(String namaPasien_1011, String keluhan_1011,
int nomor_1011) {
        this.namaPasien_1011 = namaPasien_1011;
        this.keluhan_1011 = keluhan_1011;
        this.nomor_1011 = nomor_1011;

        //pointer ke node berikutnya
        this.next_1011 = null;
    }

    //method getter
    public String getNama_2511531011() { return namaPasien_1011; }
//get nama
    public String getKeluhan_2511531011() { return keluhan_1011; }
//get keluhan
    public int getNomor_2511531011() { return nomor_1011; } //get nomor
    public Pasien_2511531011 getPointer_2511531011() { return
next_1011; } //get pointer

    //method setter
    public void setNama_2511531011() { this.namaPasien_1011 =
namaPasien_1011; } //set nama
    public void setKeluhan_2511531011() { this.keluhan_1011 =
keluhan_1011; } //set keluhan
    public void setNomor_2511531011() { this.nomor_1011 = nomor_1011;
} //set nomor
    public void setPointer_2511531011() { this.next_1011 = next_1011;
} //set pointer

    //untuk cetak data pasien
    @Override
    public String toString () {
        return "Nama Pasien: " + namaPasien_1011 + ", Keluhan: " +
keluhan_1011 + ", Nomor antrian: " + nomor_1011;
    }
}
```

B. RumahSakit_2511531011

```
package pekan5_2511531011;
```

```

import java.util.Scanner;

public class RumahSakit_2511531011 {

    //membuat head linked list
    private Pasien_2511531011 head_1011;

    //menyimpan antrian terakhir
    private int last_1011;

    //menghitung jumlah
    private int jmlPasien_1011;

    //konstruktor rs
    public RumahSakit_2511531011() {
        head_1011 = null;
        last_1011 = 0;
        jmlPasien_1011 = 0;
    }

    // insert di tail / tambah pasien di akhir linked list
    public void daftarkan_2511531011(String namaPasien_1011, String
keluhan_1011) {
        //nomor antrian bertambah
        last_1011++;
        // membuat node pasien baru
        Pasien_2511531011 baru = new Pasien_2511531011(namaPasien_1011,
keluhan_1011, last_1011);
        // jika linked list kosong
        if (head_1011 == null) {
            // pasien baru jadi head
            head_1011 = baru;
        } else {
            // traversal sampai node terakhir
            Pasien_2511531011 curr_1011 = head_1011;
            while (curr_1011.getPointer_2511531011() != null) {
                curr_1011 = curr_1011.getPointer_2511531011();
            }
            // sambungkan node terakhir ke node baru
            curr_1011.setPointer_2511531011();
        }

        jmlPasien_1011++;
        System.out.println("Pasien berhasil didaftarkan!");
        System.out.println("Nomor Antrian : " + last_1011);
    }

    // delete head / panggil pasien terdepan
    public void panggil_2511531011() {
        // cek apakah antrian kosong
        if (head_1011 == null) {
            System.out.println("Antrian kosong.");
            return;
        }

        // menyimpan pasien yang dipanggil
        Pasien_2511531011 dipanggil_1011 = head_1011;
    }
}

```

```

        // head digeser ke node berikutnya
        head_1011 = head_1011.getPointer_2511531011();
        jmlPasien_1011--;
        System.out.println("Pasien yang dipanggil:");
        System.out.println(dipanggil_1011);
    }

    // menampilkan semua antrian pasien
    public void tampilkanAntrian_1011() {
        // cek apakah list kosong
        if (head_1011 == null) {
            System.out.println("Antrian masih kosong.");
            return;
        }

        Pasien_2511531011 curr_1011 = head_1011;

        int posisi_1011 = 1;

        System.out.println("Daftar Antrian Pasien:");

        // traversal linked list
        while (curr_1011 != null) {
            System.out.println("Posisi " + posisi_1011 + " -> " +
curr_1011);
            curr_1011 = curr_1011.getPointer_2511531011();
            posisi_1011++;
        }
    }

    // mencari pasien berdasarkan nama
    public void cariPasien_1011(String namaCari_1011) {
        // cek apakah list kosong
        if (head_1011 == null) {
            System.out.println("Antrian kosong.");
            return;
        }

        Pasien_2511531011 curr_1011 = head_1011;
        int posisi_1011 = 1;
        boolean ditemukan_1011 = false;
        // traversal untuk pencarian data
        while (curr_1011 != null) {
            // pencarian tanpa membedakan huruf besar kecil
            if
(curr_1011.getNama_2511531011().equalsIgnoreCase(namaCari_1011)) {
                System.out.println("Pasien ditemukan pada posisi " +
posisi_1011);
                System.out.println(curr_1011);
                ditemukan_1011 = true;
                break;
            }
            curr_1011 = curr_1011.getPointer_2511531011();
            posisi_1011++;
        }
    }

```



```

    }
    // jika data tidak ditemukan
    if (!ditemukan_1011) {
        System.out.println("Pasien dengan nama " + namaCari_1011 + "
tidak ditemukan.");
    }
}

// cek status antrian
public void cekStatusAntrian_1011() {
    // jika antrian kosong
    if (head_1011 == null) {
        System.out.println("Antrian kosong.");
    } else {
        System.out.println("Jumlah pasien : " + jmlPasien_1011);
        System.out.println("Pasien terdepan : " + head_1011);
    }
}

// method untuk menampilkan menu
public static void tampilkanMenu_1011() {
    System.out.println("\n=== Antrian Rumah Sakit 2511531013 ===");
    System.out.println("1. Daftarkan Pasien");
    System.out.println("2. Panggil Pasien");
    System.out.println("3. Tampilkan Antrian");
    System.out.println("4. Cari Pasien");
    System.out.println("5. Cek Status Antrian");
    System.out.println("6. Keluar");
    System.out.print("Pilihan : ");
}

public static void main(String[] args) {
    Scanner input_1011 = new Scanner(System.in);
    RumahSakit_2511531011 rs_1011 = new RumahSakit_2511531011();
    int pilihan_1011;
    do {
        tampilkanMenu_1011();
        pilihan_1011 = input_1011.nextInt();
        input_1011.nextLine();
        switch (pilihan_1011) {
            case 1:
                System.out.print("Masukkan Nama Pasien : ");
                String namaPasien_1011 = input_1011.nextLine();
                System.out.print("Masukkan Keluhan : ");
                String keluhan_1011 = input_1011.nextLine();
                rs_1011.daftarkan_2511531011(namaPasien_1011,
keluhan_1011);
                break;
            case 2:
                rs_1011.panggil_2511531011();
                break;
            case 3:
                rs_1011.tampilkanAntrian_1011();
                break;
            case 4:
                System.out.print("Masukkan nama pasien yang dicari :
");
                String namaCari_1011 = input_1011.nextLine();

```

```
        rs_1011.cariPasien_1011(namaCari_1011);
        break;
    case 5:
        rs_1011.cekStatusAntrian_1011();
        break;
    case 6:
        System.out.println("Keluar dari program.");
        break;
    default:
        System.out.println("Pilihan tidak valid.");
    }
} while (pilihan_1011 != 6);
input_1011.close();
}
```

BAB III

KESIMPULAN

3.1. Kesimpulan

Penggunaan single linked list termasuk dalam penggunaan struktur data yang sederhana karena hanya menghubungkan antara node dengan setelahnya. Namun, single linked list masih memiliki kekurangan yakni tidak dapat

3.2. Saran

Karena banyaknya implementasi dari ADT, maka mahasiswa sebaiknya terus mengulang dan mempelajari setiap implementasi ADT. Mahasiswa kini hendaknya sudah mampu untuk membedakan antara array list dengan linkedlist. Untuk lebih mengasah keterampilan demi mendapatkan hasil program yang baik, hendaknya mahasiswa mampu dan terus belajar, salah satunya implementasi ADT.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, *Introduction to Algorithms*, 3rd ed., Cambridge, MA: MIT Press, 2009.
- [2] Wahyudi, *Struktur Data dan Algoritma*, Padang: Informatika, 2026
- [3] M.T. Goodrich, *Data Structures and Algorithms in Java*